TECHNICAL DESIGN RATIONALE

Alarma

Adaptive Anti-Oversleep Destination Alarm & Emergency Safety System — the reasoning behind every technology and architecture decision.

Project: Capstone & Application Development

Program: BS Information Technology — Polytechnic University of the Philippines

Platform: Android (.NET MAUI, .NET 9) · **Version:** 1.0.0

Document: Design & Technology Justification

1 Purpose of This Document

This companion to the README answers a single question for every major decision in Alarma: *why this, and not the alternative?* It documents the reasoning behind the platform, the architecture, the data and security choices, and the production build — so reviewers can evaluate the engineering judgement, not just the result.

Guiding principles. Three constraints shaped nearly every choice below: the app must work **offline-first** (commuters lose signal underground and on long routes), it must be **privacy-first** by law (Republic Act No. 10173, the Data Privacy Act of 2012), and it must remain **diagnosable in the field** where no debugger is attached.

2 Platform & Framework

Why .NET MAUI on .NET 9

CHOSEN: .NET MAUI

Why: A single C# codebase compiles to a native Android app with direct access to platform APIs (foreground services, `SmsManager`, Keystore) that the safety features depend on. The team's existing C#/.NET fluency meant no new language ramp-up, and .NET 9 brings mature AOT and IL-trimming used for the hardened Release build.

Over the alternatives: Flutter/React Native would have meant a new language and a plugin bridge for every native security call; native Kotlin would have delivered one platform at a higher cost with no benefit, since iOS is explicitly out of scope.

Why Android-only (API 26+)

CHOSEN: NET9.0-ANDROID

Why: The target users — Metro Manila jeepney, UV Express, and city-bus commuters — are overwhelmingly on budget and mid-range Android devices. API 26 (Android 8.0) as the floor covers the vast majority of in-use handsets while still giving access to modern foreground-service and notification APIs. iOS, web, and desktop are deliberately out of scope to keep the safety surface small and well-tested.

Why the MVC architecture

CHOSEN: MODEL–VIEW–CONTROLLER

Why: A clear separation between data models, XAML views, and controllers keeps the security-critical logic (validation, crypto, location handling) in testable, UI-independent classes. That separation is what makes the standalone xUnit test suite possible without any Android/MAUI dependency.

3 Data Storage & Persistence

Why SQLCipher over plain SQLite

CHOSEN: SQLITE-NET-PCL + SQLCIPHER (AES-256)

Why: All structured data — trip history, saved routes, emergency contacts, geocode cache, behavioral profiles — is sensitive. SQLCipher encrypts the entire database file at rest with AES-256, so a lost, stolen, or rooted device yields nothing readable. `sqlite-net-pcl` gives a clean ORM with **parameterized queries only**, eliminating SQL-injection surface.

Over the alternatives: Plain SQLite stores data in cleartext on disk; a flat JSON/Preferences store offers neither encryption nor query integrity.

Why split Preferences vs. SecureStorage

CHOSEN: PREFERENCES FOR SETTINGS, KEYSTORE FOR SECRETS

Why: Non-sensitive UI settings live in fast MAUI `Preferences`; every cryptographic key lives in the **Android Keystore via** `SecureStorage`. No key is ever written to Preferences or hardcoded in the binary, so decompiling the APK reveals no secrets.

One key, one home. The database, backup, and black-box keys are each 256-bit, generated by `RandomNumberGenerator`, and stored only in the Keystore (slots `alarma_db_key_v1`, `alarma_backup_key_v2`, `alarma_blackbox_key_v2`). This single rule — random keys, Keystore-resident, never hardcoded — underpins the entire cryptographic story.

4 Security & Cryptography

Security was a primary design constraint, not an afterthought. The threat model assumes an attacker with physical or filesystem access to the device.

Why AES-256-GCM everywhere encryption is local

CHOSEN: AES-256-GCM (AESGCM)

Why: GCM is **authenticated** encryption — it provides confidentiality *and* integrity in one pass. Any tampering with the ciphertext, nonce, or tag makes decryption fail before a single byte is returned. Both the encrypted backup and the forensic black-box log verify the GCM tag **before** any JSON is deserialized, closing the door on data-integrity attacks (OWASP A08).

Over the alternatives: AES-CBC would require a separate HMAC step and is easy to get wrong; using the platform's `System.Security.Cryptography` avoids hand-rolled crypto entirely.

Why a Forensic Black Box Logger

CHOSEN: ENCRYPTED, KEYSTORE-BACKED CRASH LOG

Why: Faults happen in the field — on a moving jeepney at night — where no debugger or ADB exists, and a Release build emits no `Debug.WriteLine` output. The logger seals crashes with AES-256-GCM, then on next launch decrypts the prior fault to a readable report and deletes the source (consumed once). The key is preloaded at startup so the crash handler can encrypt synchronously without awaiting, and the writer never throws — there is no crash-on-crash loop.

Why validate-before-clear on restore

CHOSEN: DEFENSIVE RESTORE DESIGN

Why: A naïve restore that wipes the database first could destroy live user data if the backup is corrupt. Alarma validates **every** record — length caps, Philippine phone-number format, coordinate bounds, quantity caps — before touching the existing database. A tampered or empty backup therefore cannot silently erase a user's contacts or routes (OWASP A04).

How the controls map to OWASP

OWASP category	Why our control addresses it
A02 Cryptographic Failures	AES-256-GCM + SQLCipher AES-256; all keys random and Keystore-resident; no hardcoded keys.
A03 Injection	Parameterized SQL only; map labels via `JsonSerializer`; search input URL-encoded; phone numbers regex-validated.
A05 Security Misconfiguration	`usesCleartextTraffic="false"` , `allowBackup="false"` , foreground service `Exported=false` .
A08 Data Integrity Failures	GCM authentication tags verified before any deserialization.
A09 Logging Failures	Encrypted forensic logging that stays readable in Release; handled exceptions routed, not swallowed.

5 Location, Mapping & Geocoding

Why an Android foreground service for tracking

CHOSEN: FOREGROUND SERVICE, EXPORTED=FALSE

Why: The destination alarm must keep computing distance even when the screen is off and the app is backgrounded — exactly what Android kills a normal background task for. A foreground `Service` with `ForegroundServiceType=TypeLocation` (required from API 34) keeps tracking alive; `Exported=false` means no other app can start or bind it. Coordinates are held in-memory as immutable values and **never logged or persisted**.

Why a PARTIAL_WAKE_LOCK

CHOSEN: SCOPED WAKE LOCK

Why: Aggressive OEM battery managers (common on the budget phones our users carry) throttle GPS delivery when the CPU sleeps, which would make the alarm miss the stop. A `PARTIAL_WAKE_LOCK` is held **only** while location providers are registered and released on stop — keeping delivery reliable without leaking the lock or draining the battery.

Why Leaflet + CartoDB in a WebView, not a native map SDK

CHOSEN: LEAFLET.JS, NO API KEY

Why: Google Maps SDK requires an API key, a billing account, and sends usage telemetry — at odds with the no-backend, privacy-first design. Leaflet with CartoDB dark tiles renders in a MAUI `WebView` under a strict Content-Security-Policy, needs **no API key and no personal identifiers**, and the dark theme suits late-night use.

Why Photon primary + Nominatim fallback + local aliases

CHOSEN: LAYERED OPEN GEOCODING

Why: Destination search is the only feature that needs the network, so it uses free, open services with no keys. Photon is fast and typo-tolerant; Nominatim is the fallback when Photon misses; and a client-side Philippine alias dictionary resolves local landmark names (e.g., colloquial stop names) that neither service indexes well. Geocode results are cached in the encrypted database to cut repeat lookups.

6 Emergency SOS

Why cellular SMS, not an internet push

CHOSEN: ANDROID SMSMANAGER

Why: An emergency is exactly when there is no Wi-Fi and no mobile data — but cellular SMS still works. Sending live coordinates directly through Android `SmsManager` means SOS functions fully offline and **never passes through any Alarma server**, so there is no backend to fail, leak, or subpoena.

Safeguards: a 2-second press-and-hold prevents accidental triggers (timer cleared on page exit), and a 30-second cooldown prevents runaway sending.

7 Production Build Hardening

The Release configuration does double duty: it shrinks and speeds up the APK *and* it raises the cost of reverse-engineering.

Setting	Why we enabled it
`RunAOTCompilation`	Compiles managed IL to native code, so there is no IL left to decompile in the shipped APK.
`AndroidEnableProfiledAot`	Profile-guided AOT prioritizes hot paths (alarm trigger, GPS callback, geocoding) for faster cold start.
`PublishTrimmed` / `TrimMode=full`	Removes unreferenced code across the whole dependency closure — smaller binary, less dead-code attack surface.
`AndroidLinkMode=SdkOnly`	Strips unused SDK/AndroidX code at the DEX level.
`DebugSymbols=false`	No symbols or sequence points, so decompilers cannot map native code back to source lines.

Why `TrimmerRoots.xml` exists. The IL trimmer cannot see types created by reflection. Because the SQLite ORM instantiates model classes reflectively, the `AlarmaApp.Models` namespace and the SQLite assemblies are explicitly rooted so full trimming does not delete classes that are only ever referenced at runtime.

8 Testing Strategy

Why a separate net9.0 xUnit project

CHOSEN: ALARMAAPP.TESTS (PLAIN NET9.0)

Why: The security-critical logic — phone-number validation, coordinate bounding, alarm-sound whitelisting, lead-time clamping, backup restore caps, AES-GCM length checks, and the speed-based multi-stage alarm — must be verifiable without spinning up an Android emulator. Targeting plain `net9.0` with no MAUI dependency keeps the suite fast and runnable in CI, which is only possible because MVC kept that logic out of the views.

9 Summary of Decisions

Area	Choice	Core reason
Framework	.NET MAUI / .NET 9	One C# codebase, native API access, mature AOT/trimming.
Platform	Android API 26+	Matches the target commuters' devices; small safety surface.
Database	SQLCipher AES-256	All local data encrypted at rest; parameterized queries.
Secrets	Android Keystore	Random keys, never hardcoded or in Preferences.
Local crypto	AES-256-GCM	Authenticated encryption; tag verified before deserialize.
Tracking	Foreground Service	Survives backgrounding; private; coordinates never persisted.
Maps	Leaflet + CartoDB	No API key, no telemetry, privacy-first.
Geocoding	Photon + Nominatim + aliases	Free, keyless, locally aware, cached.
SOS	Cellular SMS	Works offline; no backend to fail or leak.
Release	AOT + full trim	Smaller, faster, harder to reverse-engineer.